

와글와글 초원 생태계 시뮬레이션

전체 코드 설명서 — 아키텍처와 소스 해설

Python · Pygame 기반 사바나 생태계 실시간 시뮬레이션
약 4,300줄 · 8종 동물 · 4종 식물 · 자원/지형 · 시간/날씨 시스템

이 문서는 프로젝트의 모든 소스 파일을 모듈별로 설명합니다.
전체 구조 → 핵심 시스템 → 엔티티 계층 → 종별 행동 → 렌더링 → 생태 메커니즘 순서로 읽으면
코드 전체를 처음부터 끝까지 이해할 수 있도록 구성했습니다.

목차

1. 프로젝트 개요
2. 디렉터리 구조
3. 전체 아키텍처와 설계 원칙
4. 프로그램 진입점 — `main.py` · `app.py`
5. 전역 설정 — `config.py`
6. 월드: 시뮬레이션의 심장 — `world.py`
7. 환경 시스템 — `environment.py`
8. 물리 엔진(조향·충돌) — `physics.py`
9. 엔티티 기반 구조 — `entity.py` · `protocols.py`
10. 동물 공통 AI — `animal.py`
11. 식성별 동물 부모 — `carnivore` · `herbivore` · `omnivore`
12. 종별 동물 8종 상세
13. 식물 — `plants/`
14. 자원 — `resources/`
15. 지형 — `terrain/`
16. 렌더링과 카메라 — `gui.py` · `sprites.py`
17. 핵심 생태계 메커니즘(종합)
18. 조작법
19. 부록: 주요 수치표

1. 프로젝트 개요

이 프로젝트는 **아프리카 사바나(초원) 생태계를 실시간으로 흉내 내는 시뮬레이션**입니다. 사용자가 창을 열면 초원 위에서 동물들이 스스로 먹이를 찾고, 물을 마시고, 사냥하고, 도망치고, 번식하며 살아갑니다. 게임 속 시간이 흐르고 날씨와 온도가 바뀌며, 그 변화가 동·식물의 생존에 영향을 줍니다. 사람은 규칙을 직접 조작하지 않고 카메라로 관찰하거나, 속도·날씨를 바꾸거나, 개체를 클릭해 상태를 들여다볼 수 있습니다.

등장 요소

- **육식동물(Carnivore)** — 사자(Lion), 하이에나(Hyena), 대머리독수리(Bald_Eagle)
- **초식동물(Herbivore)** — 얼룩말(Zebra), 가젤(Gazelle), 코끼리(Elephant)
- **잡식동물(Omnivore)** — 미어캣(Meerkat), 혹멧돼지(Warthog)
- **식물(Plant)** — 풀(Grass), 덩굴(Bush), 아카시아나무(Acacia_Tree), 바오밥나무(Baobab_Tree)
- **자원(Resource)** — 물웅덩이(Water_Puddle), 사체(Carcass)
- **지형(Terrain)** — 평원(Plain), 동굴(Cave), 호숫가(Lake_Side)
- **환경(Environment)** — 시간·날씨(맑음/흐림/비/가름)·온도·가뭄 이벤트

기술 스택

- **언어:** Python 3 (타입 힌트 일부 사용, 한글 주석)
- **라이브러리:** Pygame 2.5+ — 창·렌더링·이벤트, 그리고 벡터 연산(**pygame.math.Vector2**)
- **외부 의존성:** **requirements.txt** 에 **pygame>=2.5** 단 하나. 가벼운 단일 의존성 구조
- **코드 규모:** 약 4,300줄, 45개 파이썬 파일

실행 방법

```
python main.py          # 일반 실행 (창이 열림)
python -m grassland      # 패키지로 실행 (동일)
python main.py --headless-steps 5000 # 창 없이 N틱만 돌려 결과만 출력(테스트용)
```

세 가지 실행 경로. headless 모드는 GUI/pygame 창 없이 시뮬레이션 로직만 빠르게 검증한다.

headless 모드는 **app.py** 가 pygame 창을 띄우지 않고 **world.update()** 만 반복한 뒤 동물·식물·자원 수를 한 줄로 출력합니다. 밸런스 실험이나 멸종 여부 검증에 쓰입니다.

2. 디렉터리 구조

프로젝트는 '진입점 → 시뮬레이션 코어 → 엔티티 → 표현(GUI)' 으로 역할이 또렷이 나뉩니다. 엔티티는 **kind**(animal/plant/resource/terrain)별로 폴더가 나뉘고, 동물은 다시 식성별로 하위 폴더(carnivores/herbivores/omnivores)를 가집니다.

```
grassland-ecosystem/
├─ main.py           # 최상위 진입점 (app.run 호출)
├─ requirements.txt  # pygame>=2.5
├─ assets/sprites/  # 배경·동물·식물·날씨 PNG 이미지
├─ grassland/       # 본체 패키지
│  └─ __main__.py   # python -m grassland 진입점
├─ app.py           # 인자 파싱 → World 생성 → GUI/headless 분기
├─ config.py        # 모든 전역 상수(숫자)만 보관
├─ world.py         # 맵 + 한 프레임 오케스트레이션 (심장부)
├─ environment.py   # 시간·날씨·온도·가뭄 이벤트
├─ physics.py       # 조향(boids) 이동 + 충돌 분리
├─ sprites.py       # PNG 로딩·크기조정·캐시
├─ gui.py           # 화면 그리기 + 카메라 (읽기 전용 렌더)
├─ entities/       # 모든 '사물'의 계층
│  └─ entity.py     # 최상위 부모 Entity (좌표·상태·이동의도)
├─ protocols.py     # Consumable / Drinkable 덕타이핑 프로토콜
├─ animals/
│  ├── animal.py    # 동물 공통 AI (스텟·먹기·도주·배회)
│  ├── carnivores/  # carnivore.py, lion, hyena, bald_eagle
│  ├── herbivores/  # herbivore.py, zebra, gazelle, elephant
│  └─ omnivores/    # omnivore.py, meerkat, warthog
├─ plants/         # plant.py, grass, bush, acacia_tree, baobab_tree
├─ resources/      # resource.py, carcass, water_puddle
├─ terrain/        # terrain.py, plain, cave, lake_side
└─ environment/    # environment_state.py (구 위치 호환 shim)
```

각 폴더의 **__init__.py** 는 하위 클래스를 한곳에서 import 할 수 있게 모아 줍니다. 예를 들어 **from grassland.entities.animals import Lion, Zebra** 처럼 짧게 불러올 수 있습니다.

3. 전체 아키텍처와 설계 원칙

계층 구조 — 누가 무엇을 책임지는가

코드는 책임이 겹치지 않도록 다섯 계층으로 나뉘어 있습니다. 위에서 아래로 '의존'하며, 아래 계층은 위를 모릅니다(예: 엔티티는 GUI를 모름).

계층	파일	책임	하지 않는 것
진입점	main.py, app.py	프로그램 시작, 모드 분기	시물 규칙·그리기
설정	config.py	모든 수치 상수 보관	로직(읽기 전용)
시물 코어	world.py, environment.py, physics.py	사물 보관, 한 프레임 진행, 시간/날씨, 이동/충돌	그리기
엔티티	entities/**	각자의 상태 + '무엇을 할지' 결정	이동의 물리·렌더
표현	gui.py, sprites.py	월드를 읽어 화면에 그림, 카메라	시물 상태 변경

핵심 설계 원칙

- **관심사 분리**: 동물은 '가고 싶은 방향'만 정하고, 실제 이동·충돌은 physics가, 그리기는 gui가 맡는다. 한 가지를 바꿔도 다른 곳이 안 깨진다.
- **데이터/표시 분리**: 엔티티는 **color-radius-action_text** 같은 표시 데이터만 들고, gui는 그것을 읽기만 한다(시물 상태를 바꾸지 않음).
- **수치의 단일 출처**: 실험하며 바꿀 모든 숫자는 **config.py** 한곳에 모은다.
- **덕 타이핑**: '먹을 수 있는 것'은 **consume()**, '마실 수 있는 것'은 **reduce_thirst()** 를 가진다(**protocols.py**). 상속이 아니라 메서드 존재 여부로 상호작용한다.

한 프레임의 데이터 흐름

게임은 매 프레임 아래 순서로 흐릅니다. **gui.run()**의 루프가 **world.update(dt)**를 부르고, 그 안에서 환경 → 엔티티 결정 → 물리 → 후처리가 차례로 일어난 뒤, 다시 gui가 그 결과를 그립니다.

```
gui.run() 루프
| 매 프레임 dt(초) 계산 (일시정지/배속 반영)
|— world.update(dt)
|   1) environment.update(dt)      # 시간·날씨·온도 진행
|   2) apply_environment_events(dt) # 가뭄이면 물 마름 / 비면 물 참
|   3) plant.update / animal.update # 각자 상태변화 + behave()로 행동 결정
|   4) physics.update(...)          # 조향 이동 + 충돌 분리 + 지형효과
|       apply_weather_effects(...)  # 더위·비가 스탯에 영향
|   5) resource.update              # 사체 부패 등
|   미어캣 엔딩 / 풀 재생 / 번식 / 사망정리 / 종료조건
|— gui.draw()                       # 월드를 읽어 화면에 렌더
```

world.update()의 5단계. '결정(엔티티)'과 '이동(physics)'이 분리된 것이 핵심.

핵심 개념 — 조향(steering) 이동 모델

이 프로젝트의 움직임은 '즉시 방향을 꺾는' 방식이 아닙니다. 동물의 AI는 **desired_velocity**(가고 싶은 속도)만 적어 두고, physics가 매 프레임 실제 **velocity**를 그쪽으로 **부드럽게 보간(steering)** 합니다. 그래서 방향이 완만히 휘고, 추격하는 포식자는 관성 때문에 급커브를 못 따라가 먹이의 지그재그에 헛돕니다. 이 한 가지 모델이 자연스러운 무리 이동·추격·회피를 모두 만들어 냅니다.

4. 프로그램 진입점 — main.py · app.py

main.py — 최상위 진입점

프로그램의 시작점입니다. 하는 일은 **app.run()** 호출 하나뿐입니다. 로직을 두지 않아 '어디서 시작하는지'가 한눈에 보입니다.

```
# main.py — 프로그램 최상위 진입점. app.run() 만 호출한다.
from grassland.app import run

if __name__ == "__main__":
    run()
```

grassland/_main.py 도 똑같이 **run()** 을 불러, **python -m grassland** 로도 실행됩니다.

app.py — 실행기(시작 절차)

명령줄 인자를 읽고, **World** 를 만들고, **GUI 실행**과 **headless 실행**을 분기합니다. 시물 규칙이나 그리기는 하지 않습니다. 눈여겨볼 점은 **pygame**을 필요할 때만 **import** 한다는 것입니다 — headless 모드에서는 창이 필요 없으므로 **gui** 를 늦게 import 해 의존성을 줄입니다.

```
def run():
    parser = argparse.ArgumentParser(description="와글와글 초원 생태계 시뮬레이션")
    parser.add_argument("--headless-steps", type=int, default=0,
                        help="창 없이 지정 횟수만큼 시뮬레이션(테스트용)")
    args = parser.parse_args()

    world = World.seed_default()      # 맵에 동·식물을 랜덤 배치

    if args.headless_steps > 0:      # 창 없이 빠르게 검증
        for _ in range(args.headless_steps):
            world.update(1 / 30)
            c = world.counts_by_name()
            print(f"Day {world.environment.day} ... | {c}")
        return

    from grassland.gui import GrasslandApp # pygame 은 창이 필요할 때만 import
    GrasslandApp(world).run()
```

app.run(): World 생성 후 모드 분기. headless는 1/30초 고정 dt로 N틱을 빠르게 돌린다.

5. 전역 설정 — config.py

실험하며 바꿀 모든 수치를 한곳에 모아 둔 파일입니다. 로직이 전혀 없고 다른 파일은 여기서 값을 읽기만 합니다. 크게 화면·시간·초기 개체수·색·날씨 틸트·스프라이트 크기로 나뉩니다.

화면과 월드(맵)

- **SCREEN_WIDTH/HEIGHT = 1280×720** — 창 기본 크기(크기 조절 가능, 최소 900×600)
- **WORLD_WIDTH = 3800** — 맵은 세로 고정·가로 스크롤. 화면보다 가로로 훨씬 넓다
- **HORIZON_Y = 260** — 지평선 높이. 이 선 위는 '하늘'(해·달·구름), 아래가 동물이 다니는 '땅'
- **WORLD_HEIGHT = SCREEN_HEIGHT - HORIZON_Y** — 동물은 지평선 위(하늘)로 올라갈 수 없다

시간

실제 1초가 게임 시간으로 얼마나 흐르는지를 정합니다. 하루는 게임 24시간이며 실제로는 약 96초입니다.

```
FPS = 60
GAME_HOURS_PER_SECOND = 0.25 # 실제 1초 = 게임 0.25h → 하루(24h)=96초
DAY_LENGTH_HOURS = 24
```

초기 개체수(SEED_COUNTS)와 미어캣 엔딩

시작할 때 배치할 동물 수, 그리고 특수 종료 시나리오인 '미어캣 엔딩'의 발동 시점을 정합니다.

```
SEED_COUNTS = {
    "Lion": 5, "Hyena": 6, "Bald_Eagle": 5,
    "Zebra": 7, "Gazelle": 7, "Elephant": 3,
    "Meerkat": 10, "Warthog": 7,
}
MEERKAT_HOME_RADIUS = 200 # 미어캣이 굴 중심에서 벗어날 수 있는 최대 거리
MEERKAT_ENDING_DAY = 4 # 이 날을 넘기면 미어캣이 거대화해 모든 것을 잠식
MEERKAT_GROW_PER_SEC = 0.03 # 거대화 진행 속도(0→1)
```

표시 크기(_sprite_display_size)

충돌용 radius 와 화면 표시 크기를 분리한 점이 중요합니다. 코끼리·바오밥은 크게, 미어캣·풀은 작게 그려 현실적인 크기비를 게임이 직접 보장합니다. 표에 없는 개체는 기본값 70(px).

개체	표시크기(px)	개체	표시크기(px)
Meerkat	30	Lion	84
Warthog / Hyena	64 / 70	Elephant	104
Gazelle / Zebra	76 / 86	Grass / Bush	70 / 95
Bald_Eagle	70	Acacia / Baobab	250 / 260
Water_Puddle / Carcass	110 / 70	Lake_Side / Cave	215 / 200

색 상수(**BACKGROUND_COLOR** 등)와 날씨 틸트(**WEATHER_TILT**: 맑음/흐림/비/가뭄별 반투명 색)도 여기 있습니다. 날씨 틸트는 gui가 화면 전체에 은은히 덧칠해 분위기를 냅니다.

6. 월드: 시뮬레이션의 심장 — world.py

World 는 '맵 그 자체'이자 **한 프레임을 지휘하는 오케스트레이터**입니다. 약 600줄로 프로젝트에서 가장 핵심적인 파일입니다. 하는 일은 네 가지입니다.

- 동물·식물·자원·지형 목록을 보관한다 (**self.animals/plants/resources/terrains**)
- **Environment**(시간·날씨)와 **PhysicsEngine**(이동) 인스턴스를 하나씩 들고 있다
- **seed_default()** 로 맵에 실제 개체를 **랜덤 배치**한다
- **update(dt)** 가 매 프레임 환경→결정→물리→후처리 순서로 전체를 진행한다

6.1 초기 배치 — seed_default()

맵을 처음 채우는 과정입니다. 순서가 중요합니다: **큰 구조물을 먼저** 배치해 자리를 차지하게 한 뒤, 풀은 그 위를 피해서 깔고, 마지막으로 동물을 놓습니다.

```
@classmethod
def seed_default(cls):
    world = cls()
    world.seed_structures() # 큰 구조물: 격자 셀에 한 개씩(겹침 0 보장)
    world.seed_grass()     # 풀: 구조물·물 위는 피해서
    world.seed_carcasses() # 사체 2~3개
    world.seed_animals()   # SEED_COUNTS 만큼 동물 배치
    return world
```

seed_structures() 는 맵을 200px 격자 셀로 나누고, 각 구조물(호숫가·동굴·나무·덤불·물웅덩이)을 '셀 한 칸에 하나씩' 놓되 셀을 벗어나지 않게 혼듭니다. 셀끼리 안 겹치니 구조물도 절대 안 겹칩니다 — **겹침 0을 보장**하면서 맵 전역에 골고루 퍼지게 하는 영리한 방법입니다. 또 맵을 좌/중/우 3구역으로 나눠 번갈아 뽑아, 같은 종류가 한쪽에 몰리지 않게 합니다.

seed_grass() 는 중앙엔 들판, 가장자리(사이드)엔 5~7포기 원형 군집으로 풀을 깔니다. **_side_spot()** 이 85% 확률로 좌·우 바깥 1/3에 위치를 잡아, 동물이 중앙에만 몰리지 않도록 먹이를 양옆에 분산합니다. **seed_animals()** 는 **SEED_COUNTS** 만큼 배치하되, 미어캣만은 동굴 근처(**_spot_near_cave**)에 둡니다.

6.2 한 프레임 — update(dt)

월드의 심장 박동입니다. 5단계가 **정해진 순서**로 일어나며, 이 순서가 곧 게임 규칙입니다.


```

def update(self, dt):
    if self.environment.ended:
        return
    self.elapsed += dt
    # 1) 환경(시간·날씨)
    if self.environment.update(dt):
        self.on_new_day()
    # 2) 환경 이벤트(가뭄이면 물 마름, 비면 물 참)
    self.apply_environment_events(dt)
    # 3) 식물·동물의 자체 변화 + 행동 결정
    for plant in self.plants:
        plant.update(self, dt)
    for animal in self.animals:
        animal.is_hidden = False          # 매 프레임 은신 초기화
        animal.interaction_target = None
        animal.update(self, dt)           # behave()로 무엇을 할지 결정
    # 4) 물리: 조향 이동 + 지형 효과 + 날씨 효과
    living = self.living_animals()
    self.physics.update(living, self.obstacles(), dt)
    self.physics.apply_terrain_effects(living, self.terrains)
    self_elephant_bounce(living)
    self.apply_weather_effects(living, dt)
    # 5) 자원 갱신 + 후처리(번식·사망·종료)
    for resource in self.resources:
        resource.update(self, dt)
    self.update_meerkat_ending(dt)
    if not self.meerkat_ending:           # 잠식 중엔 번식·풀재생 정지
        self.regrow_plants(dt)
        self.try_reproduce()
    self.flush_pending()                  # 이번 프레임 태어난 새끼 합치기
    self.check_end_conditions()

```

엔티티는 '무엇을 할지'만 정하고(3단계), 실제 이동은 4단계 physics가 한다.

왜 '결정'과 '이동'을 나눌까

3단계에서 모든 동물이 **desired_velocity**(가고 싶은 속도)를 적습니다. 4단계에서 physics가 이웃 분리·장애물 회피·가장자리 반발을 더해 실제 **velocity**로 보간합니다. 이렇게 나누면 각 동물은 '이웃이 어디 있는지'를 신경 쓰지 않고 자기 목표만 말하면 되고, 충돌·뒹침은 physics가 일괄 처리합니다. 코드가 단순해지고 움직임은 자연스러워집니다.

6.3 장애물과 '가장 가까운 무엇' 질의

obstacles() 는 동물이 통과할 수 없는 '벽' 목록을 (위치, 반지름)으로 돌려줍니다 —

나무·호숫가·물웅덩이가 벽이고, 풀·덤불은 통과 가능합니다. 동굴은 예외적으로 통과 가능합니다.

동물이 행동을 결정하려면 '가장 가까운 먹이/물/포식자/짝' 같은 **사실**이 필요합니다. world가 이를 한 무리의 **nearest_*** 메서드로 제공합니다. 모두 내부 헬퍼 **_nearest(items, pos, predicate, max_dist)** 위에 만들어졌습니다.

질의 메서드	용도
nearest_plant / nearest_bush / nearest_tree	초식·코끼리의 먹이, 매복용 덤불, 그늘 나무
nearest_water	물웅덩이 + 호숫가 중 가장 가까운 물
nearest_carcass	사체(다른 동물이 옮기는 것이 아닌 것)
nearest_preys_for	포식자의 사냥감(코끼리 제외, 초식·잡식)
nearest_predator	피식자가 보는 포식자 — 은신/스텔스 반영
nearest_weak_or_preys	잡식의 사냥감: 초식·잡식 또는 체력 50% ↓ 육식

질의 메서드	용도
nearest_same_species	번식 짝
nearest_devour_target	미어캣 엔딩 — 잡아먹을 모든 동물·식물

nearest_predator에는 게임성이 숨어 있습니다. 덩불에 숨은(**is_hidden**) 포식자는 아예 안 보이고(기습 성립), 포식자의 **stealth**가 높을수록 피식자가 인식하는 탐지 거리가 줄어듭니다 — 매복한 사자가 더 가까이 올 때까지 들키지 않는 식입니다.

7. 환경 시스템 — environment.py

좌표를 가진 '사물'이 아니라 **맵 전체에 작용하는 시스템**이라 **Entity**가 아닙니다. world가 **Environment** 인스턴스를 하나 들고 매 프레임 **update()**를 부릅니다. 시간·날씨·온도를 진행하고, 다른 시스템이 곱해 쓰는 **영향 계수**를 제공합니다.

시간과 날씨의 진행

실제 시간을 게임 시간으로 환산해 누적합니다. 게임 내 6시간마다(하루 4번) 날씨와 온도가 바뀌고, 자정을 넘기면 날짜가 1 늘어납니다. 날씨는 맑음/흐림/비/가름 중 무작위, 온도는 날씨에 따라 범위가 다릅니다.

```
WEATHER_PERIOD_HOURS = 6 # 날씨·온도가 바뀌는 주기(게임 시간)

def update(self, dt):
    previous_day = self.day
    self.change_time(dt * GAME_HOURS_PER_SECOND)
    return self.day != previous_day # 하루 넘어가면 True

def change_time(self, hours):
    self.time += hours
    self._weather_hours += hours
    while self._weather_hours >= WEATHER_PERIOD_HOURS: # 6시간마다
        self._weather_hours -= WEATHER_PERIOD_HOURS
        self.change_weather(); self.change_temp()
    while self.time >= DAY_LENGTH_HOURS: # 자정 넘기면 날짜 +1
        self.time -= DAY_LENGTH_HOURS; self.day += 1
```

동·식물에 주는 영향 계수

환경 자체는 동물을 직접 건드리지 않습니다. 대신 다른 시스템이 읽어 곱해 쓰는 계수를 내놓습니다 — '느슨한 결합'의 좋은 예입니다.

계수	맑음	흐림	비	가름	쓰임
growth_multiplier()	1.0	1.15	1.9	0.35	식물 성장·번식 속도
combat_factor()	1.0	1.0	1.05	0.85	공격력 배율(무더위엔 약화)
heat_factor()	온도 기반	—	—	—	26°C에서 0, 더울수록 ↑ → 갈증·기력소모

DroughtEvent 클래스는 가름이 지속되는 동안 **dry_up_map()**으로 물웅덩이를 조금씩 말립니다. world가 날씨가 '가름'이면 이 이벤트를 만들어 매 프레임 적용합니다.

8. 물리 엔진(조향·충돌) — physics.py

PhysicsEngine 은 동물을 한 프레임 전진시키고 겹침을 푸는 일을 합니다. 핵심은 게임 AI에서 유명한 **boids**식 조향(**steering**) 기법입니다. AI가 적어 둔 **desired_velocity** 에 매 프레임 세 가지 힘을 더해 '합성 목표 속도'를 만들고, 현재 속도를 그쪽으로 부드럽게 보간합니다.

세 가지 조향력

- ① **분리(separation)** — 너무 가까운 다른 동물에게서 멀어지는 힘. 떼가 자연히 퍼진다. 단, 지금 쫓는/상호작용하는 대상에게선 분리하지 않아 포식자가 먹이에 끝까지 붙는다.
- ② **회피(avoidance)** — 나무·물가 같은 구조물을 미리 비껴 가는 힘. 벽에 쳐박지 않는다.
- ③ **가장자리(edges)** — 맵 끝 근처에서 안쪽으로 트는 힘. 벽에 끼이지 않고 되돌아온다.

```
def _desired(self, a, animals, obstacles):
    sp = max(a.speed, 1.0)
    target = Vector2(a.desired_velocity)
    target += self._sep(a, animals) * sp          # ① 분리
    target += self._avoid(a, obstacles) * sp * 1.4 # ② 회피
    hunger = a.hunger_speed_factor() ...
    cap = max(sp, a.desired_velocity.length()) * hunger
    if target.length() > cap:
        target.scale_to_length(cap)              # 속도 상한
    target += self._edges(a) * sp * 2.2          # ③ 가장자리(상한 밖에서 더함)
    return target
```

가장자리 힘은 일부러 속도 상한 '밖'에서 더한다 — 안에서 더하면 회피력이 묻혀 0이 되기 때문.

부드러운 보간과 겹침 해소

_steer() 는 현재 속도를 목표로 **지수 보간**합니다. **agility**(민첩성)가 클수록 방향을 빨리 바꿉니다. 프레임레이트와 무관하도록 **$1 - \exp(-agility \cdot dt)$** 를 보간 계수로 씁니다.

```
def _steer(self, a, target, dt):
    a.velocity = a.velocity.lerp(target, 1.0 - math.exp(-a.agility * dt))
```

이동 후 **_separate()** 가 남은 미세 겹침을 '위치'로 살짝 밀어 분리하고, 서로를 향해 파고드는 속도 성분만 제거합니다. 속도를 통째로 죽이지 않아 벽에서 튕기거나 비벼지는 현상이 없습니다. 마지막으로 **apply_terrain_effects()** 가 동굴·호숫가처럼 '원 안에 들어오면 효과'를 주는 지형을 처리합니다.

이 조향 모델이 만들어 내는 것

① 무리가 자연스럽게 흩어지고, ② 포식자가 먹이를 끝까지 추격하며, ③ 먹이의 지그재그(evade)에 포식자가 관성으로 헛돌고, ④ 어떤 동물도 벽을 응시하며 멈춰 서거나 모서리에 끼이지 않습니다. 복잡한 경로탐색(A* 등) 없이 단순한 힘의 합만으로 이 모든 행동이 창발합니다.

9. 엔티티 기반 구조 — entity.py · protocols.py

초원의 모든 '사물'(동물·식물·자원·지형)은 공통 부모 **Entity** 를 상속합니다. 상속 트리는 다음과 같습니다.

```
Entity (좌표·속도·반지름·생사·표시데이터·이동의도)
├── Animal (스텝·먹기·도주·배회 AI)
│   ├── Carnivore — Lion, Hyena, BaldEagle
│   ├── Herbivore — Zebra, Gazelle, Elephant
│   └── Omnivore — Meerkat, Warthog
├── Plant (광합성·체력·섭취)
│   └── Grass, Bush, AcaciaTree, BaobabTree
├── Resource (amount 소모성)
│   └── WaterPuddle, Carcass
├── Terrain (원 안 효과)
│   └── Plain, Cave, LakeSide
```

Entity 를 뿌리로 한 4갈래 계층. 동물은 다시 식성별로 3갈래.

Entity — 모든 사물의 공통 뿌리

동물·식물·자원·지형이 공통으로 갖는 것을 모읍니다: 좌표(**position**)·속도(**velocity**)·충돌 크기(**radius**), 생사(**alive**)·충돌 여부(**solid**)·종류(**kind**), 그리고 화면 표시 정보(**color-render_shape-layer-action_text**). gui는 이 표시 데이터만 읽어 그립니다.

이동 모델의 핵심도 여기 있습니다. **move_toward/move_away_from/move** 는 실제 속도를 바꾸지 않고 '가고 싶은 속도'(**desired_velocity**)만 적어 둡니다. 실제 가감속·회전은 physics가 부드럽게 처리합니다.

```
def move_toward(self, target, speed):
    """target 위치 쪽으로 가고 싶다고 표시한다."""
    self.desired_velocity = self._toward(target, speed)

def stop(self):
    """가고 싶은 속도를 0으로 — physics가 부드럽게 멈춰 세운다."""
    self.desired_velocity = Vector2()
```

좌표·거리 같은 벡터 연산은 직접 구현하지 않고 **pygame.math.Vector2** 에 위임합니다(중복 제거). 각 Entity는 생성 시 고유 **id**(증가 카운터)를 받아 선택·블랙리스트 등에 쓰입니다.

protocols.py — 덕 타이핑 계약

상속이 아니라 '어떤 메서드를 가졌는가' 로 상호작용을 정의합니다. **Consumable** 은 **consume(amount)**, **Drinkable** 은 **reduce_thirst(animal)** 을 가진 객체를 뜻합니다. 덕분에 동물의 **eat()** 은 상대가 풀이든 사체든 신경 쓰지 않고 '먹을 수 있는 메서드'가 있으면 먹습니다.

10. 동물 공통 AI — animal.py

모든 동물의 공통 부모입니다(약 300줄). 스탯, 공통 행동(먹기·마시기·공격·번식·도주·배회), 그리고 '판단 → 행동'의 뼈대를 정의합니다. 종별 차이는 자식이 오버라이드합니다.

주요 스탯

스탯	의미	동작
health / max_health	체력	0 이하면 die() → 사체 생성
hunger	허기(0~100)	시간이 지나면 증가, 먹으면 감소. 높으면 먹이 탐색
thirst	갈증(0~100)	증가하다 thirst_limit 넘으면 물 찾기
stamina	기력(0~100)	이동·전투로 소모, 천천히 회복. 낮으면 느려짐
stress	스트레스	쫓기면 증가 → 포식자를 더 멀리서 감지, 회복 둔화
speed (프로퍼티)	현재 속도	기력 기반 S커브 — 지치면 느려진다
detect_range / food_range	탐지·먹이 탐지 거리	위협 감지와 먹이 감지를 분리

speed 가 단순 값이 아니라 **프로퍼티**인 점이 흥미롭습니다. 내부 **_base_speed** 에 기력 비율의 제곱근을 곱해, 기력이 떨어질수록 자연스럽게 느려집니다.

```
@property
def speed(self):
    t = self.stamina / 100.0
    return self._base_speed * (0.3 + 0.7 * t ** 0.5) # 지치면 느려짐(S커브)
```

판단의 뼈대 — update() → behave() → wander()

모든 동물은 같은 뼈대로 매 틱 행동합니다. **update()** 가 나이·기력을 갱신하고 **behave()** 를 부릅니다. **behave()** 가 '할 일을 찾으면' True를 돌려주고, 아무 일도 없으면 False → 그러면 **wander()**(배회)합니다. 자식은 보통 **behave()** 만 새로 정의하면 됩니다.

```
def update(self, world, dt):
    if not self.alive: return
    self.age += dt
    self.recover_stamina(dt)
    if not self.behave(world, dt): # 할 일이 없으면
        self.wander(world, dt)    # 어슬렁거린다

def behave(self, world, dt):
    return False                # 기본: 결정 없음(종별로 오버라이드)
```

공통 행동들

- **eat(food) / drink(source)** — 쿨다운마다 '한 입/한 모금'씩만 섭취(덱 타이핑으로 상대를 가리지 않음).
- **attack(target, world)** — 공격 쿨다운이 있어 매 프레임 죽사를 막는다(초당 ~1.4회만 타격). 날씨 **combat_factor** 로 무더위엔 약해진다.
- **seek_water_if_needed(world)** — 갈증이 한계를 넘으면 탐지범위 안 물로 이동·음수. 아주 목마르면 범위를 무시하고 찾아 나선다.
- **evade(threat_pos, ...)** — 지그재그 회피. 포식자 반대 방향에 좌우로 '번갈아' 꺾는 횡방향 힘을 더한다. 개체마다 juke 부호가 달라 무리가 사방으로 흩어진다.

- **wander(world, dt)** — 할 일이 없을 때 부드러운 곡선 배회 + 가끔 '맵 먼 곳 로밍'으로 전체 맵을 누빈다.
- **die(world)** — 죽으면 즉시 정지하고 그 자리에 사체(Carcass)를 생성한다.

자연스러운 배회의 비결 — wander()

매 프레임 독립 난수로 방향을 흔들면 '덜덜 떠는' 움직임이 됩니다. 대신 이 코드는 **각속도**(`_turn_rate`)에 랜덤 가속을 주고 지수 감쇠시킵니다. 한동안 같은 방향으로 휘다가 서서히 방향을 바꾸는 유기적인 곡선 경로가 나옵니다. 가뭄엔 로밍 목적지를 '앞이 무성한 나무'로 잡아 그늘을 찾아가게 합니다.

11. 식성별 동물 부모 — carnivore · herbivore · omnivore

동물은 식성에 따라 세 부모로 갈립니다. 각 부모는 **behave()** 를 오버라이드해 그 식성 고유의 '우선순위 판단'을 구현하고, 종별 클래스는 다시 세부를 조정합니다.

11.1 Carnivore — 포식자

고유 속성은 **stealth**(은신율), **acceleration**(추격 가속), **hunt_stamina_cost**(추격 소모)입니다. 허기·갈증이 다른 동물보다 빨리 차며, 판단 우선순위는 '코끼리 회피 → 물 → 먹이(사체·사냥) → 매복'입니다.

```
def behave(self, world, dt):
    elephant = world.nearest_named("Elephant", self.position, 90.0)
    if elephant is not None and ...: # 코끼리는 피한다
        self.move_away_from(elephant.position, self.speed); return True
    if self.seek_water_if_needed(world): return True
    if self.search_food(world, dt): return True # 사체 → 산 사냥감
    return self.ambush(world, dt) # 덫 매복
```

- **hunt

```
(prey)
```** — '예측 추격(lead pursuit)': 먹이의 현재 위치가 아니라 **position + velocity·0.35**, 즉 '갈 곳'을 노린다. 먹이가 지그재그로 꺾으면 예측이 빗나가 헛돈다.
- **ambush(world)** — 배고프면 가까운 덫에 숨어(**hide()**, **stealth** ↑) 먹이가 사정권에 들면 덮친다. 진입 110px·해제 135px의 **히스테리시스**로 경계에서 '숨었다 나왔다' 떨림을 막는다.
- **블랙리스트** — 사체를 먹으려는데 잔량이 50% 미만이면 그 사체를 **_finished_carcasses**에 등록해 다시 물려가지 않게 한다(사체 하나에 때로 쏘리는 문제 방지).

11.2 Herbivore — 초식동물

panic_range(도주 감지 거리), **is_chased**, **stress**, 그리고 도주 운(**_escape_luck**)을 다룹니다. 판단 1순위는 포식자 감지입니다. 위협이 보이면 도망치고(또는 덫에 숨고), 없으면 물·먹이를 찾습니다.

- **스트레스** → **경계**: 스트레스가 높을수록 포식자를 더 멀리서 감지(**stress·0.40** 만큼 범위 추가). 이산적 배수가 아닌 연속 증가.
- **도주 운(luck)**: 1~3초마다 삼각분포로 0.65~1.45 배율을 새로 굴려 도주 속도에 곱한다. 운 좋은 순간엔 탈출, 나쁘면 따라잡힌다 — 매번 결과가 달라지는 긴장감.
- **fight_or_flight**: 체력 70% ↑·기력 60% ↑면 맞서 싸우고(반격), 아니면 도망. 종별로 오버라이드.
- **heal**: 안 쫓기고 덜 배고프면 체력 회복(스트레스가 높으면 회복이 느려짐).

11.3 Omnivore — 잡식동물

diet_preference(0=초식~1=육식 선호)와 **aggression**(위협 시 맞설 확률)을 가집니다. 판단 우선순위는 '포식자 → 물 → 먹이'이고, 먹이 탐색에서 **aggression** 확률로 약한 사냥감을 노리거나 사체·식물을 **diet_preference**로 골라 먹습니다.

```
def flee_or_fight(self, threat, world, dt):
    # aggression 확률 + 근거리(<42)일 때만 맞서고, 아니면 도망
    if random.random() < self.aggression and self.distance_to(threat) < 42:
        self.attack(threat, world)
    else:
        self.evade(threat.position, self.speed * 1.15, dt)
```


12. 종별 동물 8종 상세

각 종은 생성자에서 자기 스탯을 정하고, 고유 행동을 추가합니다. 먼저 8종의 기본 스탯 표를 본 뒤, 종마다 '무엇이 특별한가'를 설명합니다.

| 종 (식성) | 체력 | 속도 | 공격 | 탐지 | 반지름 | 고유 행동 |
|--------------------|-----|-----|----|-----|-----|-------------------------|
| Lion 사자 (육) | 120 | 84 | 22 | 170 | 20 | roar 포효, 매복, 코끼리 회피 |
| Hyena 하이에나 (육) | 86 | 76 | 15 | 150 | 20 | 사체 탈취(steal), 빠른 가속 |
| Bald_Eagle 독수리 (육) | 58 | 122 | 10 | 300 | 14 | 비행/고도, 빈사 사냥, 분해자 |
| Zebra 얼룩말 (초) | 78 | 86 | 7 | 100 | 18 | 뒷발차기(kick), 무리 경고 |
| Gazelle 가젤 (초) | 52 | 96 | 5 | 110 | 18 | 지그재그 도주(zigzag) |
| Elephant 코끼리 (초) | 165 | 62 | 11 | 150 | 28 | 짓밟기(stomp), 넉백, 무적에 가까움 |
| Meerkat 미어캣 (잡) | 42 | 82 | 5 | 130 | 13 | 보초, 동굴 은신, 엔딩 보스 |
| Warthog 혹멧돼지 (잡) | 72 | 68 | 12 | 115 | 18 | 굴 도주(burrow), 약자 사냥 |

12.1 Lion — 사자

최강의 포식자(체력·공격력 최고). 우선순위가 정교합니다: 코끼리 회피(타이머로 3.5초 유지) → 극도 굶주림 사냥 → 물 → **포효** → 일반 사냥 → 매복. 포효(roar)는 기력 35를 쓰고 320px 안 얼룩말들을 패닉 상태로 만듭니다 (직접 안 보여도 경계 ↑). 매복 시 **hide()** 로 stealth를 0.42까지 올려 더 잘 숨습니다(갈기 덕).

12.2 Hyena — 하이에나

'사체 청소부' 컨셉입니다. 가속(**acceleration=91**)이 매우 빠르고, 낮은 허기에서도 사체를 탐합니다. 고유 행동은 **탈취(steal)**: 다른 포식자가 먹고 있는 사체를 발견하면, 그쪽으로 달려가 확률적으로 소유권(**being_eaten_by**)을 빼앗습니다. 단 하이에나끼리는 탈취 금지(무한 뺏기 순환 방지).

```
def hunt(self, prey, world, dt):
    carcass = world.nearest_carcass(preposition)
    if isinstance(carcass, Carcass) and carcass.being_eaten_by is prey:
        # 먹이가 사체를 먹는 중 → 사체로 달려가 탈취 시도
        if self.distance_to(carcass) <= ...:
            if random.random() < self.steal_prechance * (체력비율):
                carcass.being_eaten_by = self # 탈취 성공
            ...
    else:
        super().hunt(pre, world, dt) # 평범한 사냥
```

12.3 Bald_Eagle — 대머리독수리

가장 독특한 동물입니다. **분해자** 역할로 생활의 약 80%를 상공에서 선회하며, 지평선 부근 사체를 발견했을 때만 내려앉아 먹습니다. 단, **빈사(체력 20% 미만) 동물**을 탐지하면 곧장 급강하해 사냥합니다.

- **고도(altitude)**는 진짜 상태값 — 날 땐 70~200px 사이를 떠다니고(목표 고도를 가끔 새로 뽑아 서서히 다가감), 착지하면 0으로 가라앉는다. **position** 은 늘 '지면 위 투영점(그림자 위치)'이고, gui가 altitude만큼 들어 올려 그린다.
- **순찰 비행** — 할 일이 없어도 절대 멈추지 않고 방향을 주기적으로 틀며 난다(속도 벡터를 캐싱해 프레임마다 들쭉날쭉 않게).
- 탐지 거리 300으로 가장 넓고, 비행 속도(**fly_speed=188**)도 가장 빠르다.

12.4 Zebra — 얼룩말

초식동물 중 '무리 방어'가 특기입니다. 코너에 몰리면 **뒷발차기(kick)** 로 반격하고(평소 공격력을 잠깐 **kick_power**로 바꿔 때림), 도망칠 땐 **무리 경고(alert_herd)** 로 220px 안 다른 얼룩말들을 패닉시켜 함께 달아나게 합니다.

12.5 Gazelle — 가젤

가장 빠르지만(속도 96) 가장 약합니다(체력 52). 생존 전략은 오직 **지그재그 도주(zigzag)** 입니다. **zigzag_angle=52°** 가 클수록 더 예리하게 꺾여, 관성으로 못 따라오는 포식자를 혼듭니다. **agility=9** 로 민첩해 방향 전환이 빠르게 반영됩니다. 대신 지그재그는 기력 소모가 큼니다(합계 18/초).

12.6 Elephant — 코끼리

'도망치지 않는 방어자'입니다. 포식자가 가까이(90px) 오면 **짓밟기(stomp)** 로 공격하고 넉백+공중 바운스로 쫓아냅니다(10% 확률로 40 데미지 강타). 사냥 대상에서 제외되며, world의 **_elephant_bounce()** 가 닿은 사자·하이에나를 평면에서 튕겨 냅니다.

- 거의 무적 — **health** 가 프로퍼티로, 기력이 30 미만이면 받는 피해를 50% 감면한다.
- **is_hidden** 무력화 — 프로퍼티로 항상 False를 반환(덩치가 커 숨을 수 없음).
- 먹이 — 나무 잎(**eat_leaves**)을 우선으로 뜯고, 없으면 풀을 먹는다. 번식이 매우 느리다(확률 1/4).
- 예전엔 공격력이 너무 높아 포식자를 즉사시키고, 한자리에 멈춰 '벽'이 됐다. 지금은 공격력 ↓·속도 ↑로 '쫓아내는' 역할에 맞춰 조정됨.

12.7 Meerkat — 미어캣

가장 작고 약하지만(체력 42, 반지름 13) 영리합니다. 평소엔 **굴(Cave)** 반경 200px 안에서만 활동하고, 위협이 오면 동굴로 숨습니다. 할 일이 없으면 **보초(stand)** 를 서 탐지 범위를 넓히는 대신 기력을 소모합니다.

- **홈 레인지** — **wander()** 를 오버라이드해 로밍 목적지를 굴 반경 안으로만 잡고, 경계 근처에선 굴 쪽으로 방향을 튼다.
- **타임어택** — 매우 배고프거나(>75) 목마르면 포식자를 무릅쓰고 먹이·물을 확보한다.
- **미어캣 엔딩 보스(boss)** — **apocalypse** 가 켜지면 가장 가까운 동물·식물을 닥치는 대로 잡아먹는다(아래 17장).

12.8 Warthog — 흑멧돼지

잡식이며 '상황 판단형'입니다. 위협이 오면 **굴로 전력 질주해 숨고(burrow)**, 최대 5초 머뭅니다(쿨타임 5초). 단, 위협이 빈사 상태(체력 50%↓)면 도망 대신 맞서 싸우고, 평소 배고프면 약한 사냥감(**nearest_weak_or_prey**)을 노립니다. 굴 근처에서 적이 아주 가까우면 숨는 대신 그 자리에서 반격하기도 합니다.

13. 식물 — plants/

식물은 **kind="plant"**, **solid=False**(통과 가능), **layer=1**(바닥층)인 Entity입니다. 공통 부모 **Plant** 는 체력·광합성·섭취를 정의하고, 4종이 이를 확장합니다.

Plant — 공통 부모

photosynthesize() 로 매 프레임 체력을 회복하되 **날씨 계수(growth_multiplier)**를 곱합니다 — 비 오면 쑥쑥, 가뭄이면 시듭니다. **consume(amount)** 는 동물이 먹을 때 실제 먹힌 양을 돌려주고, 체력이 0이 되면 죽습니다.

| 종 | 체력 | 광합성/s | 반지름 | 특징 |
|------------------|-----|-------|-----|--|
| Grass 풀 | 55 | 1.4 | 16 | 비 올 때 추가 성장. 씨앗으로 번식(world가 재생) |
| Bush 덤불 | 80 | 0.8 | 30 | 은신처! hide_entity 로 동물을 숨기고 스트레스 ↓ |
| Acacia_Tree 아카시아 | 130 | 0.4 | 34 | 가시(thorn) 피해 + 잎 시스템, 그늘 제공 |
| Baobab_Tree 바오밥 | 180 | 0.6 | 42 | 저장수분 + 잎 시스템, 그늘 제공 |

- **Bush(덤불)** — 초식동물이 숨고, 포식자가 매복하는 핵심 은신처. **current_foliage**(잎 양)가 많을수록 은신 효과가 크다. 먹히면 잎이 줄어든다.
- **Acacia(아카시아)** — **on_eaten_by()** 로 잎이 남아 있을 때 먹은 동물에게 가시 피해(4)를 준다. 8초마다 잎을 재생하고, 코끼리는 **eat_leaves()** 로 잎만 뜯어 먹는다.
- **Baobab(바오밥)** — 가장 크고 튼튼. 가뭄 때 **provide_shade()** 로 동물 스트레스를 낮춰 더위를 피하게 한다.

나무의 두 가지 역할

아카시아·바오밥은 단순 먹이가 아닙니다. ① 동물이 통과 못 하는 **장애물(벽)** 이고, ② 가뭄 때 **has_foliage()**(잎 충분)면 **그늘**을 제공해 더위 피해를 막아 줍니다. world의 **apply_weather_effects**가 가뭄에 그늘 나무 아래 동물을 보호합니다.

14. 자원 — resources/

자원은 **amount**(양)를 가진 소모성 Entity입니다. 공통 부모 **Resource** 는 **consume()**·**regenerate()**·**delete()** 를 정의합니다.

Carcass — 사체

동물이 죽으면 그 자리에 생깁니다(**amount=85**). 먹을수록 양이 줄고, 0이 되면 부패 후 사라집니다. 아무도 안 먹어도 **decomposition_timer**(30초) 뒤부터 서서히 분해됩니다. **being_eaten_by** 는 현재 먹는 포식자를 가리켜 **하이에나** **탈취**의 핵심 연결고리가 됩니다. gui는 남은 양에 따라 carcass0~3 이미지를 골라 그립니다.

Water_Puddle — 물웅덩이

reduce_thirst() 로 한 모금씩 갈증을 줄입니다(물도 함께 줌). 비가 오면 **fill_rain()** 으로 차오르고, 가뭄엔 **DroughtEvent** 가 발생합니다. 비 올 때 world가 가끔 새 웅덩이를 만들기도 합니다.

15. 지형 — terrain/

지형은 **solid=False**(통과 가능)이고 '원 안에 들어오면 효과'를 주는 Entity입니다. 공통 부모 **Terrain** 의 **contains(entity)** 로 범위를 판정하고, **give_effect(entity)** 로 효과를 줍니다(자식이 오버라이드).

| 지형 | 역할 | 효과 |
|---------------|------|--|
| Plain 평원 | 배경 | 효과 없음. 맵 전체를 덮는 바탕 |
| Cave 동굴 | 은신처 | 미어캣·흑멧돼지가 들어가면 숨겨짐(is_hidden). 모든 동물 통과 가능 |
| Lake_Side 호숫가 | 큰 수원 | 갈증 해소(웅덩이보다 물이 훨씬 많음). 비 오면 불어남 |

LakeSide 는 동물이 통과 못 하는 **장애물**이기도 해서, 물가에서 마시되 물 속으로 들어가진 않습니다.

Cave 는 물리 충돌에서 제외하고 **apply_terrain_effects** 의 효과로만 처리합니다 — 두 시스템이 동시에 밀면 경계에서 '비벼지는' 현상이 생기기 때문입니다.

16. 렌더링과 카메라 — gui.py · sprites.py

표현 계층입니다. world를 읽기만 해서 화면에 그립니다(카메라 제외 시물 상태를 바꾸지 않음). **gui.py** 가 약 1,060줄로 가장 길지만, 게임 로직은 전혀 없고 '무엇을 어디에 어떻게 그릴까'만 다룹니다.

16.1 sprites.py — 이미지 로더와 캐시

PNG를 디스크에서 읽어 캐시합니다. 매 프레임 디스크를 읽으면 느리므로, 한 번 읽은 원본과 크기 조정 결과를 dict에 저장합니다. 파일 규칙은 <개체 name 소문자>.png(예: Lion→lion.png). 이미지가 없으면 None을 돌려주고 gui가 자주색 점으로 폴백합니다.

- **get_sprite(name, size)** — 긴 변이 size가 되도록 비율 유지 축소(캐시).
- **trimmed(name, size, mode)** — 동물 전용. 투명 여백을 잘라낸 '실제 그림' 기준으로 크기를 맞춘다. 프레임마다 캔버스·여백이 달라도 화면상 크기가 일정하고 발밑이 정확하다. 기본 mode는 **geom**(면적 기준), 독수리만 **height**(날갯짓 때 가로폭만 변하므로).

16.2 카메라와 좌표 변환

맵은 세로 고정·가로 스크롤입니다. 카메라는 가로(**camera.x**)만 움직이고, **world_to_screen()** 이 월드 좌표를 화면 좌표로 바꿉니다 — 세로엔 지평선 높이(**field_top**)를 더해 '하늘 띠 아래'부터 그립니다. 마우스 드래그로 카메라를 움직이고, 짧은 클릭은 개체 선택으로 구분합니다.

16.3 그리는 순서(draw)

그리기 순서가 곧 '무엇이 무엇을 가리는가'입니다.

```
def draw(self):
    self.screen.fill(BACKGROUND_COLOR) # 단색 초록(그림 없을 때)
    has_bg = self.draw_background()    # 배경 그림 타일링(거울 타일)
    self.draw_sky(...)                 # 해·달·구름(패럴랙스)
    self.draw_world()                  # 동물·구조물 — 깊이 정렬
    self.draw_weather_tint()           # 날씨 반투명 덧칠
    self.draw_rain()                   # 비 입자
    self.draw_ui()                     # 좌하단 정보 패널 + 미니맵
    self.draw_selection_panel()        # 선택 개체 속성(우상단)
    self.draw_weather_tooltip()        # 날씨 아이콘 툴팁
```

16.4 2.5D 깊이감 — 발밑 정렬과 그림자

tap다운이지만 입체감을 줍니다. '서 있는' 것들(동물·나무·동굴)은 **발밑 y 좌표 기준으로 정렬**해 앞의 것이 뒤를 가립니다(**anchor="bottom"**). '누운' 것들(호숫가·웅덩이·사체)은 중심 기준으로 먼저 깔립니다.

- **그림자** — 떠오른 동물(독수리)의 원래 발밑에 열린 타원을 그려 '뜨' 느낌을 준다. 높이 떨어수록 작고 멀어진다.
- **들어 올리기(lift)** — 고도(altitude) + 통통 튀는 모션(bounce)만큼 동물을 위로 올려 그린다. 독수리의 비행, 코끼리 짓밟기 바운스가 여기서 표현된다.
- **바라보는 방향** — 실제 이동 속도(velocity.x)를 따라 좌우 반전. 떨림 방지용 쿨다운이 있다.
- **머리 위 체력바** — 비율에 따라 길이와 색이 초록→노랑→빨강으로 바뀐다.
- **풀(Grass)** 만든 깊이 정렬에서 빼 항상 동물·나무 뒤에 깎다 — 안 그러면 큰 동물 몸통을 뚫고 솟아 보인다.

16.5 애니메이션 상태 기계

동물 그림은 **action_text**(현재 행동)에 따라 프레임이 바뀝니다. 클래스 변수 **ANIMATIONS** 가 동물별로 '행동 → 상태 → (프레임 목록, 프레임당 초)'를 정의합니다. **_frame()** 이 타이머를 진전시켜 지금 그릴 프레임을 고릅니다.

- 행동→상태 매핑이 있으면 그걸 쓰고, 없으면 이동 속도로 walk/idle을 고른다.
- 실제 파일이 없는 프레임은 자동으로 걸러지고, 다 없으면 기본 이미지(idle)로 폴백한다.
- **_ANIM_MIN_HOLD** — 독수리 급강하·코끼리 짓밟기처럼 순간 트리거되는 동작은 최소 지속 시간을 뒤, 한 프레임만 번쩍이고 사라지지 않게 한다.

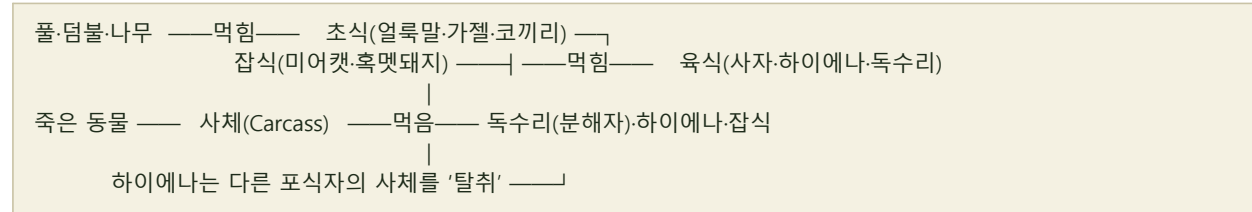
16.6 하늘·날씨·UI

- **하늘** — 해·달이 게임 시간을 따라 호를 그리며 뜨고, 하늘색이 낮/노을/밤으로 보간된다. 구름은 월드 좌표를 가지고 카메라보다 천천히 흐른다(**패럴랙스**). 날씨별 목표 구름 수에 맞춰 서서히 늘었다 줄었다 한다.
- **날씨 표현** — 화면 전체에 반투명 틴트(WEATHER_TINT)를 덧칠하고, 비 올 땐 빗줄기 입자를 그린다. 날씨 아이콘에 마우스를 올리면 효과 요약 툴팁이 뜬다.
- **정보 패널** — 좌하단에 Day·시각·온도·배속·날씨·동물 수를 보여주고, 하단에 전체 맵 **미니맵**(동물=색점, 현재 화면 영역=흰 테두리)을 그린다. 접기/펼치기 가능.
- **선택 패널** — 개체를 클릭하면 우상단에 좌표·체력·허기·갈증·기력·스트레스·행동 등 상세 속성을 띄운다.

17. 핵심 생태계 메커니즘(종합)

앞에서 파일별로 본 조각들이 어떻게 하나의 생태계로 맞물리는지 정리합니다.

17.1 먹이사슬



초식↔육식의 기본 사슬에, 사체를 둘러싼 분해자·청소부·탈취가 얹힌다.

17.2 스탯 순환

동물의 삶은 네 스탯의 줄다리기입니다. **hunger-thirst** 는 시간이 지나면 차오르고, 일정선을 넘으면 먹이·물을 찾아 나섭니다. **stamina** 는 이동·전투·도주로 닳고 천천히 회복되며, 낮으면 속도가 떨어집니다. **stress** 는 쫓길 때 쌓여 경계심을 키우지만 회복을 늦춥니다. **health** 가 0이 되면 죽어 사체가 됩니다.

17.3 번식과 개체수 균형

`world.try_reproduce()` 가 매 프레임 낮은 확률로 번식을 시도합니다. 안전·포만 상태이고 같은 종 짝이 가까이 있어야 합니다. 핵심은 **개체수가 줄수록 번식 확률이 올라가는 density_factor** 로, 멸종 직전 회복력을 줍니다.

- 번식 상한: 육식 6마리, 초식 12마리(코끼리는 더 느림).
- 번식 확률(프레임당): 육식 0.0010, 초식 0.0018, 코끼리 0.0005 — 여기에 `density_factor`(최대 +1.5배)를 곱한다.
- 허기·갈증이 52를 넘거나 쫓기는 중이면 번식하지 않는다.

밸런스 튜닝 기록

초기 설정에서 '3사자+5하이에나+4독수리 vs 7얼룩말+7가젤' 구성은 피식자가 붕괴(멸종)했습니다. 포식자 허기 증가율을 1.5→1.3/초로 낮추고, `density_factor`로 회복력을 준 뒤 180초 10회 시험에서 피식자 멸종 0회를 확인했습니다. 밸런스를 조정할 땐 `SEED_COUNTS`, 번식 상한·확률, 허기율을 먼저 보세요.

17.4 날씨가 생태에 미치는 영향

| 날씨 | 식물 | 동물 체력/기력 | 물 | 전투력 |
|------------|-------------|--------------|------------------|-------------|
| 맑음 sunny | 보통(×1.0) | 더위 시 갈증 ↑ | — | 보통 |
| 흐림 cloudy | 약간 ↑(×1.15) | 체력 +0.5/s | — | 보통 |
| 비 rain | 쑥쑥(×1.9) | 체력 +0.5/s | 호수·웅덩이 보충, 새 웅덩이 | 소폭 ↑(×1.05) |
| 가뭄 drought | 급감(×0.35) | 그늘 밖 체력·기력 ↓ | 물 고갈 진행 | 하락(×0.85) |

가뭄엔 동물이 '잎이 무성한 나무 그늘' 아래로 모입니다 — 그늘 안이면 더위 피해를 받지 않습니다. **d** 키로 날씨를 강제로 바꿔 이 변화를 직접 관찰할 수 있습니다.

17.5 미어캣 엔딩

이 시뮬레이션의 특수 종료 시나리오입니다. 게임 **MEERKAT_ENDING_DAY**(기본 4일)를 넘기면, 미어캣들이 서서히 거대해지며(크기·체력·공격·속도·탐지 모두 폭증) 모든 동물·식물·나무를 잡아먹기 시작합니다.

- 엔딩 전엔 크기만 아주 조금씩 커진다(스탯 변화 없음).
- 엔딩 발동 후엔 **_grow**(0→1)에 따라 화면 크기 최대 ~4.5배, 체력 최대 ~492, 거의 무적(굶지도 지치지도 않음).
- **boss()** 행동으로 가장 가까운 대상을 닥치는 대로 먹어 치운다. 잠식 중엔 풀 재생·일반 번식이 멈춰 잠식이 완성될 수 있다.
- 미어캣 외 모든 동물과 식물이 사라지면 시뮬레이션이 '미어캣 엔딩'으로 종료된다.

18. 조작법

| 입력 | 기능 |
|---------------|---|
| 마우스 드래그 | 카메라 좌우 이동(가로 스크롤) |
| 개체 클릭 | 동물·지형·자원·식물 선택 → 우상단에 상세 속성. 같은 개체 재클릭 시 해제 |
| 빈 곳 클릭 | 그 지점의 월드 좌표를 좌하단에 표시 |
| Space | 일시정지 / 재개 |
| ← / → | 시물 배속 변경 (0.25 · 0.5 · 1 · 2 · 4배) |
| D | 날씨 강제 전환(맑음→흐림→비→가뭄 순환) |
| E | 미어캣 엔딩 강제 발동(날짜를 엔딩일로 점프) |
| R | 시뮬레이션 재시작(맵 새로 생성) |
| Y / N (종료 화면) | 재시작 / 종료 |
| 좌하단 ▲/▼ | 정보 패널 접기 / 펼치기 |

19. 부록: 주요 수치표

초기 개체수 (config.SEED_COUNTS)

| 종 | 수 | 종 | 수 |
|----------------|----|--------------|---|
| Lion 사자 | 5 | Zebra 얼룩말 | 7 |
| Hyena 하이에나 | 6 | Gazelle 가젤 | 7 |
| Bald_Eagle 독수리 | 5 | Elephant 코끼리 | 3 |
| Meerkat 미어캣 | 10 | Warthog 혹멧돼지 | 7 |

주요 전역 상수 (config.py)

| 상수 | 값 | 의미 |
|-----------------------|------------|-------------------------|
| SCREEN_WIDTH × HEIGHT | 1280 × 720 | 창 기본 크기 |
| WORLD_WIDTH | 3800 | 맵 가로 길이(가로 스크롤) |
| HORIZON_Y | 260 | 지평선 높이(위=하늘, 아래=땅) |
| FPS | 60 | 초당 프레임 |
| GAME_HOURS_PER_SECOND | 0.25 | 실제 1초=게임 0.25h (하루 96초) |
| MEERKAT_HOME_RADIUS | 200 | 미어캣 활동 반경 |
| MEERKAT_ENDING_DAY | 4 | 미어캣 엔딩 발동일 |
| 번식 상한 (육식 / 초식) | 6 / 12 | 종당 최대 개체수 |

이 문서는 프로젝트의 모든 소스 파일(약 4,300줄)을 모듈별로 해설했습니다. 코드를 직접 읽을 때는 **config.py(수치) → world.py(흐름) → animal.py(판단 뼈대) → 종별 클래스(고유 행동)** 순서로 따라가면 전체 구조가 가장 빠르게 머릿속에 그려집니다.